

MTH2008

Scientific computing

Second semester

1st lecture

Daniilo Roccatano - Marco Pinna

Office : INB3129

Email: Mpinna@lincoln.ac.uk

Summary Semester A

Type	Occurence	date	contribution
Coursework	1	3/3/2025	20%
Logbook	1	20/03/2025	10%
portfolio test	1	28/04/2025	20%

Padlet

<https://uol.padlet.org/mpinna2/scientific-computing-l5c5c5v5fmvv72l0>

Assessments

- During semester B, you will also perform one 1 Time Constrained Test (Portfolio Test) that will be used to review the learning progress of the class. It will consist of questions and exercises covering the part of the program until the week before the test.
- Each student must also continue to keep a logbook. Only the semester B part of the logbook needs to be submitted.
- There will also be 1 coursework assignment used for a continuous assessment throughout the module delivery.

Contribution Assessments per Semester

Semester A = 50%

Semester B = 50%

Summary of semester A contents

C++ language

- Preprocessors
- Variables
- Control statements (e.g. for-loop, while-loop, do-while-loop)
- C++ functions
- Conditional statements (if-else, switch statement)
- Arrays
- Pointers first introduction
- Strings
- Streams (I/O on files)

NUMERICAL METHODS

- Calculation of simple statistical properties of a set of numbers
- Series and Taylor Series

Overview Semester B

NUMERICAL METHODS AND ALGORITHMS

- Random Numbers
- Sorting algorithm
- Numerical Differentiation
- Root of a equations
 - Bisection Method
 - Newton Method
 - Secant Method
- Numerical Differentiation
- Numerical Integration
- Numerical integration of Ordinary Differential Equation (first order)
- Linear Algebra
 - Product of scalar with vectors/matrix
 - Product of two matrix

Random number generators (RNG)

A random number generator should also have the following properties

- 1.Repeatability. Generate the same sequence for testing purpose
- 2.Sequence length. The randomness of the generated number should hold for very long sequences of number.
- 3.Speed. The generator should be very fast.

For high-security systems, generating genuinely random keys is vital. To meet this need, specialized physical devices employ unpredictable processes like quantum phenomena or radioactive decay. These devices enhance security by ensuring keys lack discernible patterns, thwarting potential attacks based on predictability.

Truly Random Numbers

However, these generators come with some problems

- A simulation that uses a truly random number is not repeatable.
- These quantum random number generators are not seeded, and each simulation will differ.
- Any interesting event or phenomenon may not be reproducible.
- Consequently, such devices are unnecessary for the majority of scientific computing applications.

Pseudo random Number Generators

Software-generated random numbers lack true randomness and are referred to as

Computational Random Number Generators, specifically Pseudo-Random Number Generators (PRNGs).

Although PRNGs exhibit properties similar to truly random numbers, they are not identical. However, these properties are sufficient for the majority of computational algorithms that use random numbers.

PRNG algorithms properties

- **The algorithm should be fast.** Individual computations are inexpensive, but a simulation may require millions of random numbers.
- **Portable to different computers** – ideally to different programming languages. This ensures the program produces the same results.
- **Have a sufficiently long cycle.** The cycle length, or period, represents the length of a random number sequence before previous numbers begin to repeat in an earlier order.
- **Replicable.** Given the starting point, it should be possible to generate the same set of random numbers completely independent of the system that is being simulated.
- **Closely approximate the ideal statistical properties** of uniformity and independence.

PRNG algorithms

- Computer systems often come supplied with a RNG built in.
- While these are fast and easy to use, they may not be very random, have short repeat sequences and may have strong correlation patterns.

PRNG Algorithms: Linear Congruential Generators

A frequently employed algorithm in numerous built-in Random Number Generators (RNGs) is the Linear Congruential Generator (LCG).

A sequence of random values X_n can be generated using the recurrence relation:

$$X_{n+1} = (aX_n + c) \bmod m$$

with well-chosen integer values a, c, m

The period of the sequence is at most m but it can be shorter for unfortunate choices of the previous numbers X_0 is called the seed

The selection of the values for a, c, m , and X_0 drastically affects the statistical properties and the cycle length.

- If $c \neq 0$ then it is called mixed congruential method
- When $c = 0$ it is called multiplicative congruential method

PRNG Algorithms: Linear Congruential Generators

**m=10,
a=1,
c=7**

1

$$(1 \cdot 1 + 7) \% 10 = 8$$

$$(1 \cdot 8 + 7) \% 10 = 5$$

$$(1 \cdot 5 + 7) \% 10 = 2$$

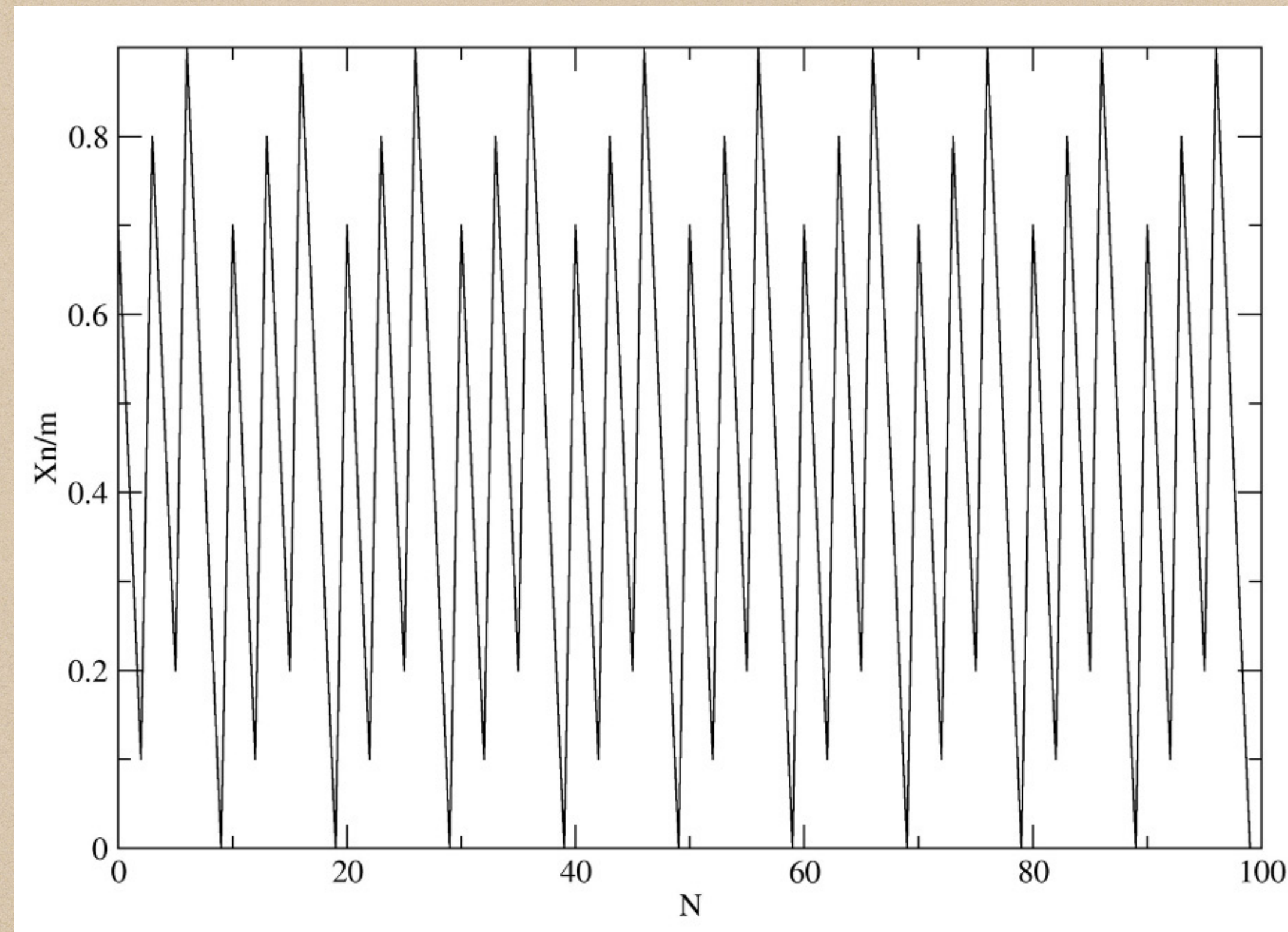
$$(1 \cdot 2 + 7) \% 10 = 9$$

$$(1 \cdot 9 + 7) \% 10 = 6$$

$$(1 \cdot 6 + 7) \% 10 = 3$$

$$(1 \cdot 3 + 7) \% 10 = 0$$

...

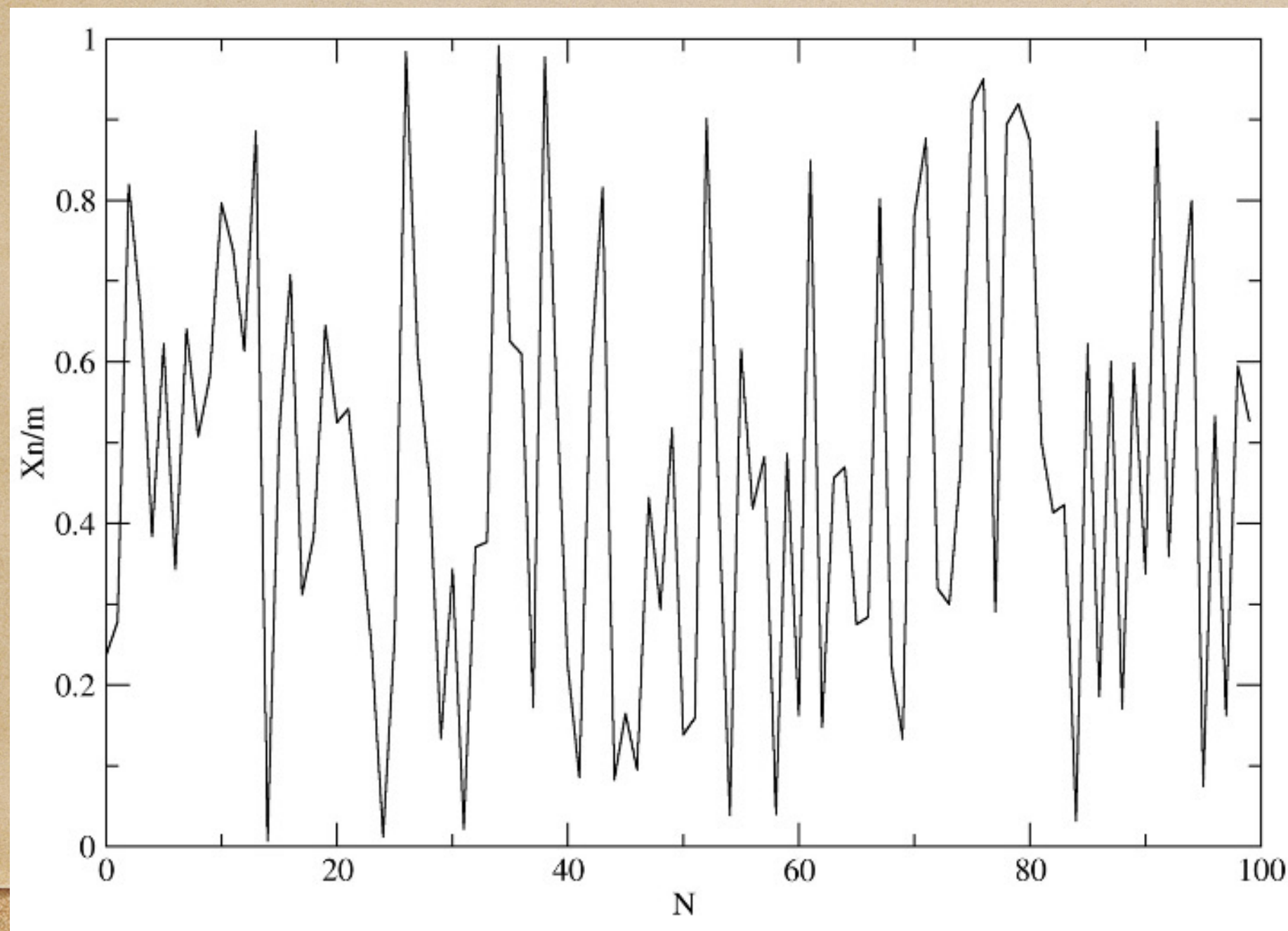


PRNG Algorithms: Linear Congruential Generators

Compilers for libraries of numerical programs often employ various parameters for the Linear Congruential Generator (LCG). These parameters determine the behavior of the LCG and influence the statistical properties of the generated pseudo-random numbers.

As an example, the influential Numerical Recipes* book recommends the following values:
 $m=2^{32}$ $a=1664525$ $c=1013904223$

An example of 100 random numbers generated using this set of parameters is given in the figure



* Press et al. Numerical recipes in C++,
Cambridge University Press, 7 feb 2002

Pseudo random Number Generators

```
#include <iostream>
#include <cstdlib> // Needed for rand() and srand()
#include <ctime>   // Needed for time()
using namespace std;

int main()
{
    int b, c;
    double num[10]; // Declaration of the array for 10 numbers
    double a;
    int size;

    size = 10; // Number of elements to sort out

    // Seed the random number generator with the current time
    //srand(static_cast<unsigned int>(time(0)));

    // Initialization of the array
    for (c = 0; c < size; c++)
        num[c] = ((double)rand() / RAND_MAX ) * 100; // Random function with range [0,99]

        //num[c] = ((double)rand() / RAND_MAX) - 0.5; // Random function with range [-0.5, 0.5]

    // Displaying the array on the screen
    cout << "Original numbers are: " << endl;
    for (c = 0; c < size; c++)
        cout << num[c] << endl;
    cout << endl;

    return 0;
}
```

rand1.cpp

Pseudo random Number Generators

```
#include <iostream>
#include <cstdlib> // Needed for rand() and srand()
#include <ctime>    // Needed for time()
using namespace std;

int main()
{
    int b, c;
    double num[10]; // Declaration of the array for 10 numbers
    double a;
    int size;

    size = 10; // Number of elements to sort out

    // Seed the random number generator with the current time
    srand(static_cast<unsigned int>(time(0)));

    // Initialization of the array
    for (c = 0; c < size; c++)
        //num[c] = (rand() % 100); // Random function with range [0, 99]
        num[c] = (rand() % 100) + 1; // Random function with range [1, 100]

    // Displaying the array on the screen
    cout << "Original numbers are: " << endl;
    for (c = 0; c < size; c++)
        cout << num[c] << endl;
    cout << endl;

    return 0;
}
```

rand2.cpp

what are the main
differences between the
two codes?

Pseudo random Number Generators

N = number of counts in each interval between 0 and 1

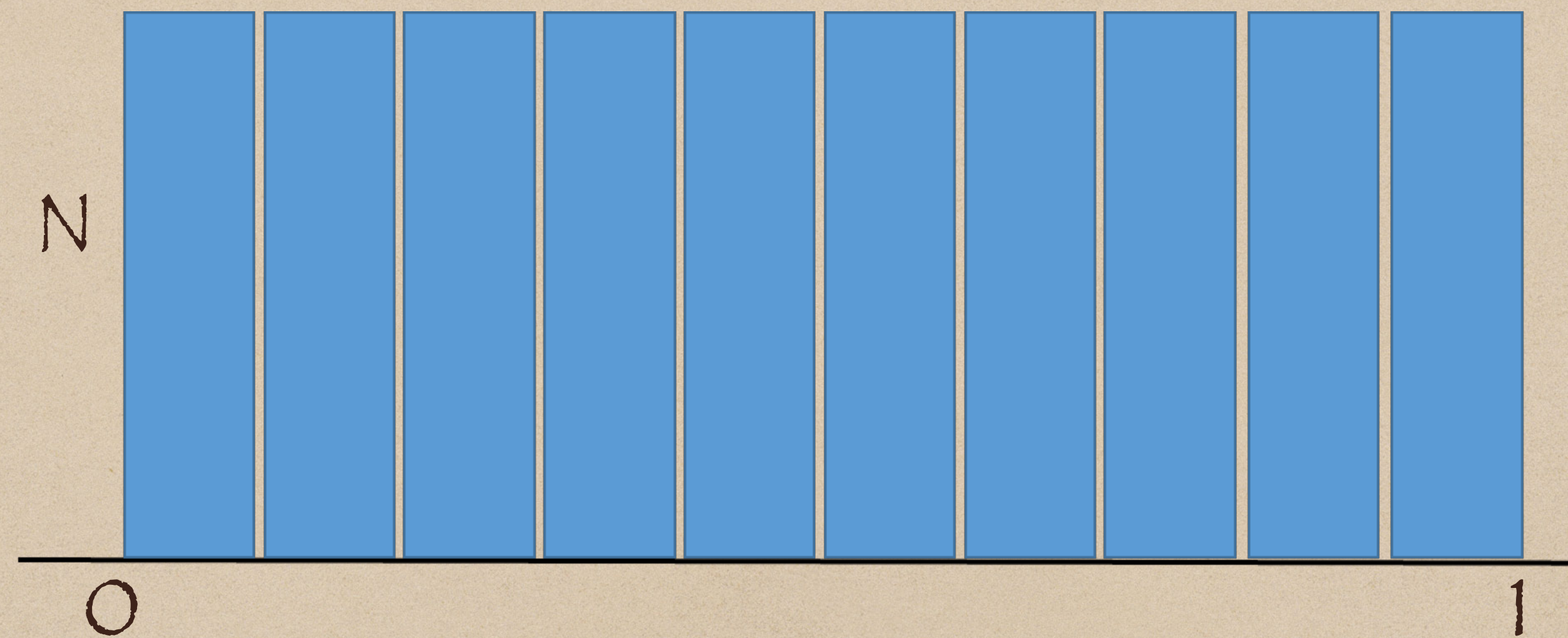


Fig 1. Perfect random number generator

Pseudo random Number Generators

N= number of counts in each interval between 0 and 1

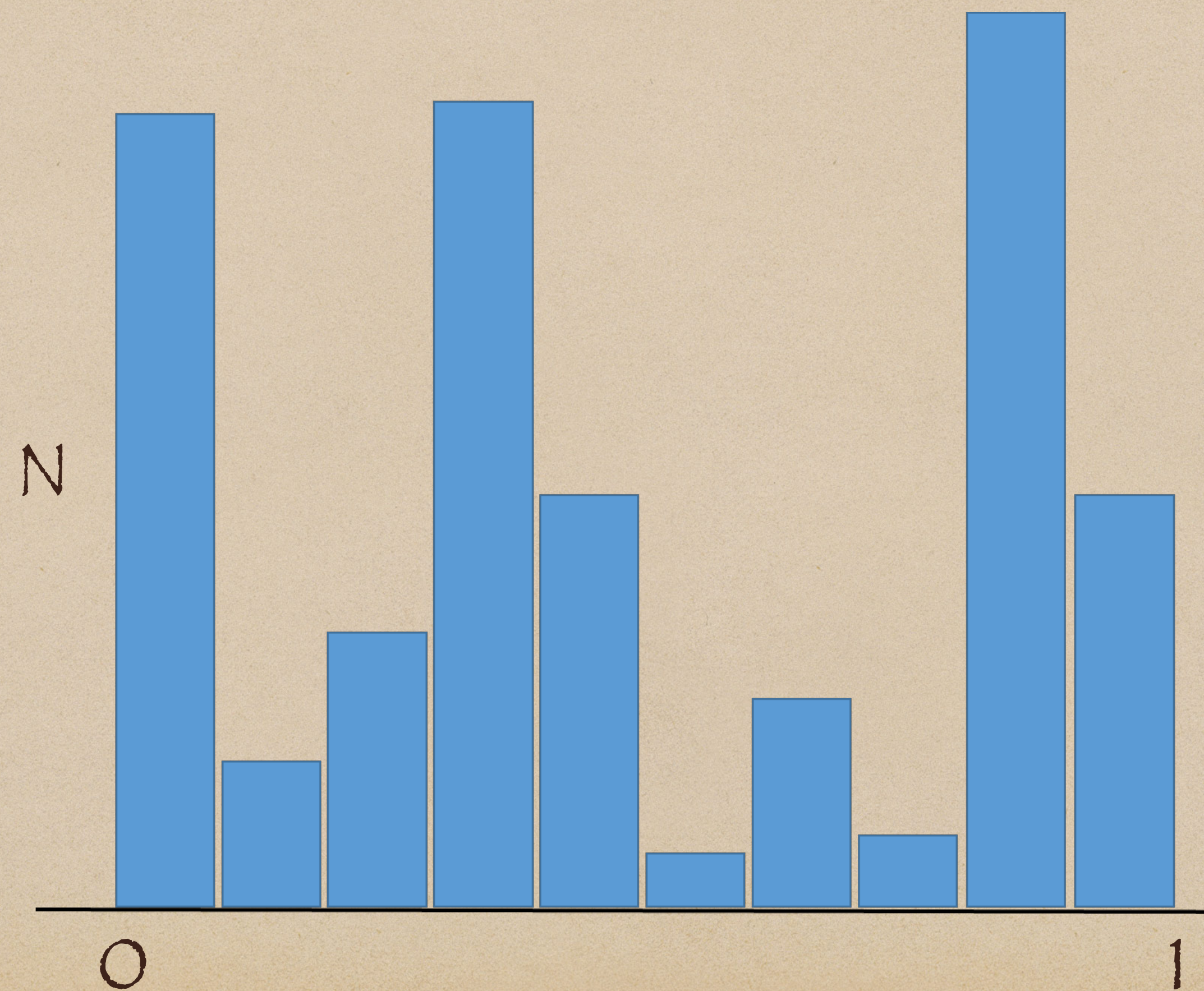


Fig 2.

Pseudo random Number Generators

N= number of counts in each interval between 0 and 1

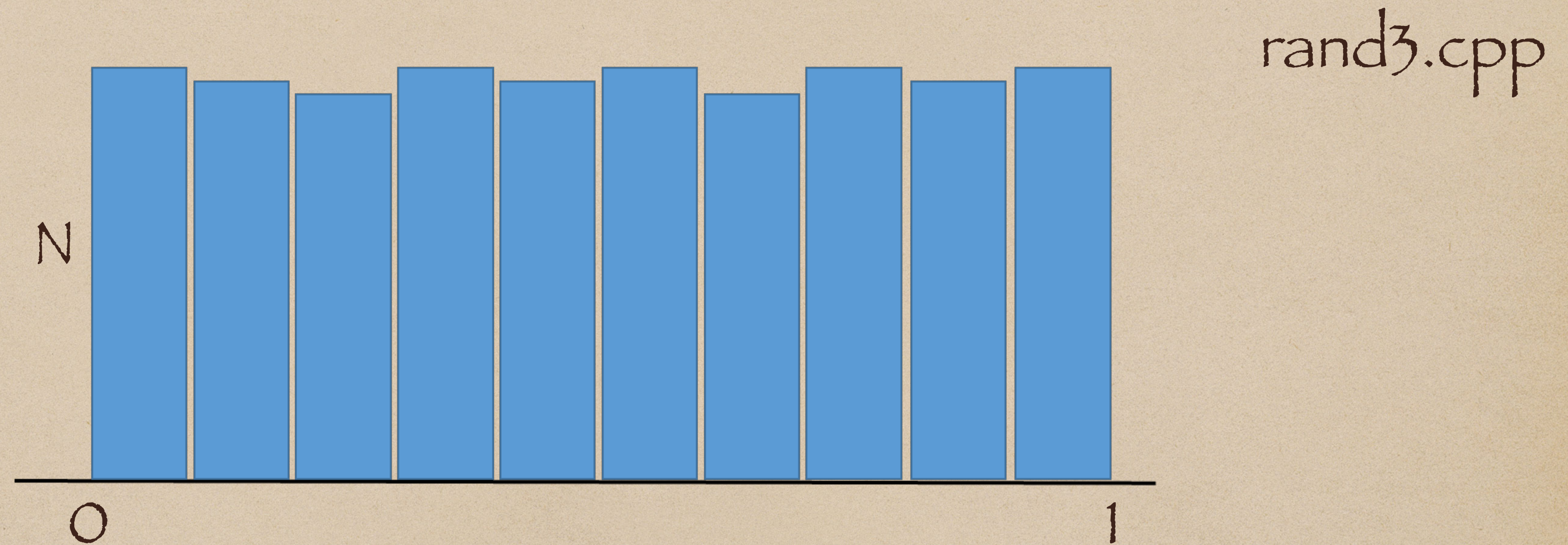


Fig 3. Real random number generator

Modern random generators

```
#include <iostream>    // For input and output
#include <cstdlib>      // For rand(), srand(), and RAND_MAX
#include <ctime>        // For time()
#include <random>       // Modern random number generation
using namespace std;
...
...
...
// Modern random number generator
    random_device rd;                // Random device for seeding
    mt19937 gen(rd());                // Mersenne Twister engine
    uniform_real_distribution<double> dist(0.0, 1.0); // Uniform distribution in [0.0, 1.0)

// Initialization of the array with random numbers between 0 and 1
for (c = 0; c < size; c++) {
    num[c] = dist(gen); // Generate random doubles in [0.0, 1.0)
}
```

rand4.cpp

rand5.cpp -to build histograms

LAB exercises TASK 1.1

WRITE A C++ FUNCTION THAT GENERATES RANDOM NUMBERS USING THE MIXED LINEAR CONGRUENTIAL GENERATOR ALGORITHM

LAB task 1.2 - Lotto

- a) WRITE A C++ PROGRAM THAT EXTRACT 6 DIFFERENT RANDOM NUMBERS IN THE RANGE 1-50 AND PRINT THE LUCKY NUMBERS ON THE SCREEN.
- b) EXTEND THE PROGRAM BY EVALUATING THE NUMBER OF EXTRACTIONS REQUIRED TO WIN WITH 2, 3, 4, 5 and 6 MATCHING NUMBER OUT OF 6 of YOUR CHOICE.

Questions?